КАК РАБОТАЕТ СТАТИКА И ДИНАМИКА

Подробное техническое объяснение работы static- и dynamic-слоев проекта EH/Unwind Analyzer, включая runtime function tables, JIT-сценарии, Windows Debug API, границы возможностей и корректную формулировку для защиты.

Вопрос	Краткий ответ
Может ли dynamic проверить появление таблиц при runtime/JIT?	Да, если исполняемый путь приводит к вызовам RtlAddFunctionTable или RtlInstallFunctionTableCallback. Проект ставит breakpoints на эти API и сохраняет аргументы вызовов как runtime evidence.
Что это доказывает?	Что во время выполнения процесс зарегистрировал дополнительную unwind/function map, которой нет как обычной статической .pdata-карты PE-файла.
Чего это не доказывает?	Это не доказывает автоматически, что код именно JIT, не проверяет каждую запись runtime table и не гарантирует обнаружение путей, которые не были исполнены в данной dynamic-сессии.

1. Главная модель проекта

Проект нельзя корректно описывать как простую программу, которая "читает .pdata". Более точная формулировка: это система структурного анализа и динамической валидации Windows x64 EH/Unwind-метаданных. Static-слой строит карту, заявленную PE-файлом. Dynamic-слой проверяет отдельные факты, которые появляются только во время выполнения процесса.

Ключевая мысль: .pdata в PE-файле - это не вся вселенная unwind-информации. Это статическая часть карты. Windows x64 действительно использует статические RUNTIME_FUNCTION записи из Exception Directory, но процесс во время выполнения может зарегистрировать дополнительные function tables для памяти, которая не была описана обычной .pdata на диске. Поэтому static отвечает на вопрос "что обещает файл", а dynamic отвечает на вопрос "какие дополнительные факты появились при реальном запуске".

Эта разница принципиальна для защиты. Если сказать, что проект "полностью видит JIT", это будет неверно. Правильная научная формулировка такая: динамическая часть проекта обнаруживает и документирует регистрации runtime function tables через поддерживаемые Windows API. Такие регистрации характерны для JIT-компиляции, runtime-generated code, интерпретаторов, защитных оболочек и некоторых рантаймов, но сам факт вызова API еще не является строгим доказательством конкретной причины.

Static и dynamic в проекте не конкурируют. Static шире покрывает файл, потому что он может пройти все записи .pdata и все доступные UNWIND_INFO, даже если соответствующий код никогда не исполнится. Dynamic уже по охвату, потому что видит только один или несколько runtime-маршрутов, но зато его сигнал сильнее в отношении фактического поведения: если handler получил управление или runtime table API реально вызван, это уже не гипотеза из файла, а событие живого процесса.

2. Мини-глоссарий: адресация и уровни наблюдения

Термин	Что означает в проекте
PE-файл	Portable Executable, исполняемый формат Windows. Для проекта это исходный объект static-анализа: bytes, DOS header, NT headers, секции, data directories, Exception Directory.
ImageBase	Базовый виртуальный адрес, от которого Windows обычно размещает образ модуля в памяти. В файле многие адреса хранятся не как абсолютные VA, а как RVA относительно ImageBase.
RVA	Relative Virtual Address, относительный виртуальный адрес внутри образа. BeginAddress, EndAddress и UnwindInfoAddress в RUNTIME_FUNCTION являются RVA, а не file offset.
VA	Virtual Address, адрес в памяти процесса. Для главного модуля часто VA = runtime image base + RVA. Dynamic работает именно с VA, потому что breakpoint ставится в живой процесс.
File offset	Смещение в файле на диске. Чтобы прочитать байты UNWIND_INFO из файла, проект переводит RVA в file offset через таблицу секций.
.pdata	Обычное имя секции, где часто лежит массив RUNTIME_FUNCTION. Формально важна не только секция с именем .pdata, а Exception Directory из PE Optional Header.
.xdata	Обычное имя области, где часто лежит UNWIND_INFO. На практике она может находиться в .rdata или другой read-only области, поэтому проект проверяет mapping и permissions.
RUNTIME_FUNCTION	12-байтовая запись x64 exception directory: BeginAddress, EndAddress, UnwindInfoAddress. Она связывает диапазон кода функции с метаданными восстановления стека.
UNWIND_INFO	Структура, описывающая, как отменить действия пролога функции: какие nonvolatile registers сохранены, сколько bytes выделено на стеке, есть ли handler или chain.

Термин	Что означает в проекте
UNWIND_CODE	2-байтовый slot внутри UNWIND_INFO. Важный нюанс: одна логическая операция может занимать несколько slots, поэтому парсер обязан учитывать дополнительное место под operands.
Handler	Адрес обработчика исключений, связанный с UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER. Static видит RVA handler-a, dynamic может поставить breakpoint на соответствующий VA.
Runtime function table	Таблица RUNTIME_FUNCTION, зарегистрированная уже в процессе через API вроде RtlAddFunctionTable. Она может описывать JIT/generate code и отсутствовать как обычная .pdata на диске.
JIT	Just-in-time compilation: генерация машинного кода во время выполнения. Для корректного unwind на x64 такой код часто регистрирует runtime function table, но сам проект не классифицирует каждую регистрацию как JIT без контекста.

3. Static: что именно анализируется

Static-слой начинает с файла, а не с процесса. Он читает байты PE-образа и последовательно восстанавливает структурную карту EH/Unwind. В этой карте есть несколько уровней: контейнер PE, таблица секций, Exception Directory, массив RUNTIME_FUNCTION, отдельные UNWIND_INFO записи, handler RVA, CHAININFO и набор эвристических признаков.

Первый уровень static-анализа - контейнерный. Проект проверяет, что файл похож на PE x64, что заголовки читаемы, что секции не требуют чтения за пределами файла, что виртуальные диапазоны секций не образуют опасных конфликтов и что можно корректно переводить RVA в file offset. Это базовая защита от неправильного разбора: если адресация сломана, любой дальнейший вывод становится сомнительным.

Второй уровень - Exception Directory. Проект берет data directory для exception information, проверяет RVA и Size, мапит эту область в file offset и читает массив RUNTIME_FUNCTION. Каждая запись должна занимать 12 bytes. Если размер directory не кратен 12, это структурная проблема: таблица не может быть прочитана как корректный массив записей.

Третий уровень - диапазоны функций. Для каждой RUNTIME_FUNCTION записи проект анализирует BeginAddress, EndAddress и UnwindInfoAddress. BeginAddress должен быть меньше EndAddress, диапазон должен лежать внутри image, а UnwindInfoAddress должен вести в читаемую область. Также проверяются сортировка, overlaps, duplicate BeginAddress и выходы за пределы образа.

Четвертый уровень - чтение UNWIND_INFO. Здесь важны Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister, FrameOffset, массив UNWIND_CODE, дополнительные operand slots, padding до четного числа slots, tail с handler RVA или CHAININFO. Именно на этом уровне проект учитывает типичные ошибки ручного парсинга: путаницу CountOfCodes, сдвиг на padding, неверное чтение handler tail и цепочек CHAININFO.

Пятый уровень - эвристика. После структурного чтения проект не просто печатает поля. Он группирует признаки по семействам evidence: table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance. Затем строит score, severity, confidence, evidence level и рекомендации manual review. Confidence здесь не является статистической вероятностью; это инженерная уверенность в интерпретации набора признаков.

4. Static: пошаговый маршрут проекта

Шаг	Действие	Смысл
1	Чтение файла	Проект загружает bytes PE-файла в буфер. Смысл: все последующие операции должны быть проверяемыми по границам, иначе поврежденный файл сможет заставить анализатор читать несуществующие данные.
2	Проверка PE x64	Проверяются PE headers и архитектура. Смысл: x64 EH/Unwind устроен иначе, чем x86 SEH, поэтому проект не должен молча применять x64-правила к неподходящему формату.
3	Разбор секций	Строится карта section RVA -> file offset и section permissions. Смысл: один и тот же RVA нельзя понимать без секций; permissions нужны для признаков вроде writable metadata или handler in non-executable section.
4	Поиск Exception Directory	Из Optional Header берется directory с exception information. Смысл: проект анализирует не имя секции .pdata само по себе, а официальную PE-карту, которая указывает, где таблица реально находится.
5	Чтение RUNTIME_FUNCTION	Directory читается как массив 12-байтовых записей BeginAddress, EndAddress, UnwindInfoAddress. Смысл: это базовая таблица соответствия function range -> unwind metadata.
6	Проверка диапазонов	Проект проверяет Begin < End, image bounds, сортировку, overlaps, duplicate begins. Смысл: Windows lookup ожидает осмысленную карту функций; конфликты ухудшают восстановление caller context.

Шаг	Действие	Смысл
7	Переход к UNWIND_INFO	UnwindInfoAddress трактуется как RVA и переводится в file offset. Смысл: это один из самых частых источников человеческих ошибок, потому что RVA нельзя читать как смещение в файле напрямую.
8	Чтение header UNWIND_INFO	Читаются Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister/FrameOffset. Смысл: эти поля определяют дальнейший формат записи и допустимость tail.
9	Чтение UNWIND_CODE	Проект читает CountOfCodes logical slots, учитывая, что отдельные UWOP требуют дополнительных slots. Смысл: если принять один UWOP за один slot всегда, tail будет прочитан со сдвигом.
10	Учет padding	Количество slots на диске выравнивается до четного числа. Смысл: handler RVA или chained RUNTIME_FUNCTION начинается после выровненного массива, а не после последнего человечески видимого UWOP.
11	Разбор tail	По Flags определяется, есть ли EHANDLER/UHANDLER или CHAININFO. Смысл: один и тот же хвост после codes имеет разное значение в зависимости от flags.
12	Следование CHAININFO	Если включено follow-chain, проект ограниченно идет по цепочке, контролирует глубину и ищет loop. Смысл: chain может скрывать итоговые свойства primary function, но бесконечная цепочка является проблемой.
13	Сводка effects	Из UWOP собираются fixed_stack_bytes, pushed_nonvol_count, saved registers, frame. Смысл: отчет показывает не только сырой набор slots, а интерпретацию unwind effects.
14	Эвристика и evidence model	Подсчитываются признаки, severity, score, confidence, evidence level, manual validation steps. Смысл: анализатор превращает низкоуровневые нарушения в читаемый инженерный вывод.
15	Формирование отчетов	Пишутся JSON, CSV, Ghidra CSV и HTML. Смысл: разные потребители получают разные формы данных: машинная интеграция, ручная проверка, презентационная сводка.

5. Static: важные нюансы .pdata и .xdata

RUNTIME_FUNCTION содержит RVA, а не VA и не file offset. Это означает, что UnwindInfoAddress сначала должен быть интерпретирован как адрес внутри образа, затем переведен через таблицу секций в смещение файла. Проект делает именно такую последовательность. В терминах защиты: он не читает .xdata "по имени секции", а строит проверяемую адресную цепочку.

CountOfCodes означает количество 2-байтовых slots в массиве UNWIND_CODE, но не количество человеческих операций в простом смысле. Например, UWOP_ALLOC_LARGE может занимать два или три slots. Проект вычисляет slots_for_code для каждой операции и проверяет, что операция не выходит за CountOfCodes. Если выходит, разбор этой записи прекращается с диагностикой, потому что дальнейший tail уже нельзя считать надежным.

Padding учитывается через выравнивание количества slots до четного числа. В коде это выражено как slots_even = (count + 1) & ~1. Смысл правила: handler RVA или CHAININFO tail должен начинаться после выровненного массива, иначе 32-битное поле будет прочитано с ошибочным смещением. Это ровно тот нюанс, который вручную легко пропустить.

Flags определяют смысл хвоста. Если установлен UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, tail начинается с handler RVA. Если установлен UNW_FLAG_CHAININFO, tail содержит chained RUNTIME_FUNCTION-like entry. Одновременная комбинация CHAININFO с EHANDLER/UHANDLER некорректна, и проект отмечает ее как аномалию. Это важно, потому что без проверки flags нельзя понять, какие bytes после codes являются handler-ом, а какие - chained record.

Language-specific handler data в текущем проекте не парсится глубоко. Проект читает handler RVA и может проверить placement handler-а, но не строит универсальный разбор C scope table или C++ EH data. Это корректная граница: формат language-specific data зависит от конкретного handler-а и runtime-а. Универсальный парсер без знания handler-а давал бы ложную точность.

Проект не выполняет дизассемблирование .text и не сверяет каждый UWOP с фактическими инструкциями пролога. Он выводит структурную интерпретацию UNWIND_INFO и выявляет нарушения самой metadata. Для будущего улучшения можно добавить disassembly-based semantic layer, который будет проверять, что UWOP_PUSH_NONVOL RBX соответствует push rbx в пределах SizeOfProlog, а UWOP_ALLOC_SMALL соответствует sub rsp, imm.

6. Static: что он может утверждать и что не должен утверждать

Вопрос	Корректный вывод static
Файл содержит корректно читаемую Exception Directory?	Да или нет, с указанием RVA, Size, section mapping и причины ошибки при невозможности чтения.
Какие функции описаны в .pdata?	Диапазоны BeginAddress..EndAddress, связанный UnwindInfoAddress, секции и размер диапазона.
Есть ли структурные повреждения?	Да, если нарушены bounds, кратность, сортировка, overlaps, CountOfCodes, tail, chain, handler bounds или другие формальные свойства.
Есть ли подозрительные признаки?	Да, в виде findings с severity, family, score contribution и confidence. Это инженерные эвристики, а не ML-классификатор.
Можно ли сказать, что код точно вредоносный?	Нет. Проект выявляет anomalies и suspicious metadata patterns. Окончательная интерпретация требует reverse engineering контекста.
Можно ли сказать, что runtime не добавит новую карту?	Нет. Static видит файл. Runtime function tables могут появиться только при выполнении, поэтому это зона dynamic.

7. Dynamic: зачем он нужен

Dynamic-слой нужен не потому, что static "плохой", а потому, что static по определению ограничен файлом. Если процесс во время выполнения регистрирует новые function tables, ставит vectored exception handlers, меняет маршрут обработки исключений или реально попадает в handler, этого нельзя надежно увидеть только из PE-байтов на диске.

В контексте проекта dynamic - это динамическая валидация. Он запускает target как debuggee, получает Windows Debug API события, ставит breakpoints на интересующие адреса и сохраняет evidence: регистрации handlers, вызовы runtime function table API, исключения, first-chance/second-chance стадию, попадания в handlers и короткий RIP trace после первого handler hit.

Dynamic не заменяет static. Он берет статическую карту как вход: например, список handler RVA из UNWIND_INFO. Затем переводит эти RVA в runtime VA через base address главного модуля и ставит breakpoints. Поэтому dynamic в проекте не является отдельной независимой программой; он является вторым слоем доказательной системы поверх статической карты.

Главная ценность dynamic для темы .pdata/.xdata такая: он показывает, когда статическая карта файла перестает быть полной картой unwind-информации процесса. Если target вызывает RtlAddFunctionTable или RtlInstallFunctionTableCallback, в отчете появляется runtime evidence: процесс регистрирует дополнительную таблицу функций. Это именно ответ на тезис "pdata в PE-файле - не вся вселенная unwind-информации".

8. Dynamic: технический маршрут запуска

Шаг	Действие	Смысл
1	Static сначала	Перед dynamic проект уже прочитал PE, RUNTIME_FUNCTION и UNWIND_INFO. Это нужно, чтобы знать handler RVA и контекст статических findings.
2	Сбор handler RVA	Из успешно разобранных UNWIND_INFO выбираются уникальные handler_rva. Смысл: если handler реально получит управление, breakpoint подтвердит runtime-достижимость этого адреса.
3	CreateProcessA	Target запускается через CreateProcessA с флагом DEBUG_ONLY_THIS_PROCESS. Смысл: analyzer становится debugger-ом только для этого процесса и получает stream debug events.
4	CREATE_PROCESS_DEBUG_EVENT	Проект получает base address главного модуля. Смысл: теперь статические RVA можно перевести в runtime VA как base_main + RVA.
5	Обновление module list	Проект собирает список загруженных modules. Смысл: API breakpoints ставятся по VA внутри конкретных modules, например ntdll.dll или kernel32.dll.
6	API breakpoints	Проект находит адреса AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable, RtlInstallFunctionTableCallback и ставит INT3 breakpoints.
7	Handler breakpoints	Для handler-ов из UNWIND_INFO ставятся breakpoints по base_main + handler_rva. Смысл: проверить не только наличие handler RVA, но и факт попадания в handler.
8	Debug loop	Через WaitForDebugEvent проект обрабатывает CREATE_PROCESS, LOAD_DLL, UNLOAD_DLL, EXCEPTION и EXIT_PROCESS события. Смысл: dynamic наблюдает процесс последовательно, а не по одному снимку.
9	Exception events	Обычные исключения логируются с exception_code, address и first_chance/second_chance. Для сохранения естественной обработки target получает DBG_EXCEPTION_NOT_HANDLED.

Шаг	Действие	Смысл
10	Breakpoint events	При попадании в наш breakpoint проект очищает INT3, откатывает RIP на адрес breakpoint, читает thread context и достает аргументы API из регистров x64 calling convention.
11	Runtime evidence	Для каждого события создается DynamicEvent: type, module, thread_id, address, exception_code, first_chance, arg0..arg3. Смысл: отчет хранит не лозунг, а конкретные наблюдаемые факты.
12	Trace after handler hit	После первого handler hit проект может включить single-step trace до trace_steps адресов. Смысл: получить короткий RIP маршрут после попадания в handler.
13	Остановка	Сессия завершается по EXIT_PROCESS или timeout. Отчет фиксирует ok/timed_out/target_exited/instrumentation_complete, чтобы читатель понимал качество dynamic-наблюдения.

9. Dynamic: какие Windows API контролируются

API	Почему важен для проекта
AddVectoredExceptionHandler	Регистрация VEH, то есть vectored exception handler. Такой handler может получать управление до обычной SEH-диспетчеризации и использоваться как легальный runtime-механизм или как часть exception-based control flow.
AddVectoredContinueHandler	Регистрация VCH, то есть vectored continue handler. Он участвует на стадии продолжения после обработки исключения и может влиять на дальнейший control flow.
SetUnhandledExceptionFilter	Установка top-level unhandled exception filter. Это поздний обработчик, который получает управление, когда исключение осталось необработанным обычными механизмами.
RtlAddFunctionTable	Регистрация массива RUNTIME_FUNCTION во время выполнения. Это ключевой сигнал, что unwind-карта процесса расширяется runtime-таблицей, которой может не быть в .pdata файла.
RtlInstallFunctionTableCallback	Регистрация callback-механизма для function table. Вместо готового массива процесс может предоставить callback, который будет выдавать function entries для диапазона адресов.

В текущей реализации именно эти API являются поддерживаемыми точками наблюдения. Это означает, что вопрос "видит ли проект runtime tables" надо понимать буквально: он видит их регистрацию через RtlAddFunctionTable и RtlInstallFunctionTableCallback. Если runtime использует другой механизм, например growable function table API, прямой поддержки этого конкретного вызова в текущем списке breakpoints нет. Такое расширение является реалистичным улучшением, но не текущим заявленным функционалом.

При попадании на RtlAddFunctionTable проект читает аргументы по соглашению вызова Windows x64: RCX содержит pointer на таблицу, RDX - количество entries, R8 - base address. В отчете это сохраняется как table, count, base. Это позволяет сказать не просто "dynamic table detected", а "процесс вызвал RtlAddFunctionTable с такими аргументами".

При попадании на RtlInstallFunctionTableCallback проект читает RCX, RDX, R8, R9: table identifier, base address, length и callback. В отчете это сохраняется как id, base, length, callback. Такой сигнал особенно важен для runtime-generated code: вместо статической таблицы процесс может предоставить callback, который Windows будет использовать для поиска unwind-информации в указанном диапазоне.

10. JIT и runtime function tables: точная формулировка

JIT означает just-in-time compilation: генерацию машинного кода во время выполнения. Для Windows x64 важна не сама идея "код сгенерирован", а вопрос: сможет ли система корректно выполнить stack unwind, exception dispatch и walk stack для этого кода. Если код находится в памяти, но не описан обычной .pdata PE-файла, runtime должен предоставить Windows другую карту функций.

Эта другая карта и есть runtime function table. Она может быть добавлена через RtlAddFunctionTable или организована через RtlInstallFunctionTableCallback. Поэтому dynamic-слой проекта проверяет не абстрактный "JIT вообще", а конкретный системный след: вызов API регистрации дополнительной function table. Это научно корректный, воспроизводимый и проверяемый критерий.

Если в dynamic report есть событие rtl_add_function_table, можно утверждать: в данной execution session target зарегистрировал дополнительную RUNTIME_FUNCTION table. Это значит, что статическая .pdata из PE-файла не является полной картой unwind-информации процесса. Но нельзя автоматически утверждать: "это точно JIT". Возможны benign runtime stubs, язык runtime-а, exception support, protector, packer, VM dispatcher или другое runtime-generated code.

Если в dynamic report нет rtl_add_function_table и нет rtl_install_function_table_callback, это не доказывает отсутствие JIT или runtime tables во всех условиях. Это означает только, что в данной сессии, при данном входе, при данном execution path и при текущем наборе breakpoints такие вызовы не наблюдались. Dynamic

всегда зависит от достигнутого маршрута исполнения.

11. Dynamic events: как читать arg0..arg3

Event type	Значение полей
exception	address = адрес исключения; exception_code = код исключения; first_chance = стадия first/second chance; arg0..arg3 не используются как основные смысловые поля.
veh_registration	address = адрес API breakpoint; arg0 = first/order; arg1 = handler VA. После этого проект пытается поставить breakpoint на handler.
vch_registration	address = адрес API breakpoint; arg0 = first/order; arg1 = handler VA. После этого проект пытается поставить breakpoint на VCH handler.
uef_registration	address = адрес API breakpoint; arg0 = handler VA. Это top-level unhandled exception filter.
rtl_add_function_table	address = адрес вызванного API; arg0 = pointer на FunctionTable; arg1 = EntryCount; arg2 = BaseAddress; arg3 не используется.
rtl_install_function_table_callback	address = адрес вызванного API; arg0 = TableIdentifier; arg1 = BaseAddress; arg2 = Length; arg3 = Callback.
unwind_handler_hit	address = VA handler-а из UNWIND_INFO, куда реально попало исполнение. Это сильный dynamic-сигнал, потому что handler не только существует в metadata, но был достигнут.
veh_handler_hit / vch_handler_hit / uef_handler_hit	address = VA соответствующего runtime handler-а. Эти события подтверждают, что зарегистрированный обработчик получил управление.
process_exit	arg0 = exit code target-а. Важно для оценки полноты dynamic-сессии: процесс завершился сам или был остановлен timeout-ом.
timeout	arg0 = timeout_ms. Это не finding по binary, а признак неполной dynamic-сессии.

12. First-chance и second-chance

First-chance exception - это первая стадия наблюдения исключения отладчиком. Когда в процессе возникает exception, Windows сначала сообщает debugger-у: "возникло исключение, еще не завершена обычная обработка". Если debugger не берет исключение на себя, система продолжает штатный dispatch: VEH, SEH/EH mechanisms, frame handlers, фильтры и другие механизмы.

Second-chance exception - это стадия, когда исключение не было обработано обычными механизмами и снова попало к debugger-у уже как потенциально завершающее. Для анализа это важный сигнал: first-chance может быть нормальным управляемым механизмом, а second-chance чаще говорит, что процесс не смог обработать исключение и может завершиться аварийно.

Проект сохраняет counters first_chance_seen и second_chance_seen. Это не просто статистика. Это помогает отделить штатную exception-based логику от аварийного поведения. Например, controlled exception, который вызывает VEH и затем продолжается, может быть first-chance evidence. А second-chance вместе с target crash показывает, что runtime route не был успешно обработан.

13. Что означает instrumentation_complete

Dynamic-сессия полезна только тогда, когда понятно ее качество. Поэтому проект хранит не только события, но и технические counters: modules_observed_peak, module_refresh_failures, module_list_truncations, breakpoint_set_attempts, breakpoint_set_failures, pending_rearm_failures, breakpoint_rearm_failures, dropped_events.

Instrumentation complete означает, что инструмент не зафиксировал критичных проблем с постановкой и сопровождением breakpoints. Если instrumentation_complete = false, найденные события остаются реальными, но отсутствие событий нельзя трактовать как сильное доказательство отсутствия поведения.

Например, если breakpoint на нужный API не поставился, dynamic не мог наблюдать соответствующий вызов.

Это важно для защиты: проект не скрывает техническое качество эксперимента. Он показывает, насколько полно была проведена runtime-инструментация. Такой подход профессиональнее, чем просто написать "dynamic OK" или "dynamic failed" без объяснения причин.

14. Dynamic и handler hits

Static может увидеть handler RVA в UNWIND_INFO, но это еще не значит, что handler реально получит управление при запуске. Handler может быть диагностическим, может относиться к редко достижимому коду, может быть частью нормального runtime-а или может вообще не достигаться при текущем input.

Dynamic делает следующий шаг: он берет handler RVA, вычисляет VA как base_main + handler_rva и ставит breakpoint. Если execution доходит до этого адреса, появляется unwind_handler_hit. Это сильнее статического факта, потому что runtime доказал достижимость handler-а в конкретной сессии.

Но handler hit тоже нужно интерпретировать аккуратно. Сам факт handler hit не означает автоматически вредоносность. Он означает, что exception handling route реально использовался. Дальше реверсер должен смотреть static context: где расположен handler, сколько функций на него ссылается, есть ли writable/executable anomalies, есть ли RaiseException/int3/guard pages, как меняется RIP после обработки.

Проект помогает именно тем, что не заставляет реверсера вручную искать все handler RVA, ставить breakpoints и вести счет. Он автоматизирует этот рутинный слой и сохраняет evidence в JSON/HTML. Но глубокую семантическую интерпретацию control flow пока оставляет специалисту.

15. Dynamic и короткий RIP trace

RIP trace в проекте - это короткая выборка instruction pointer после первого handler hit. Технически проект включает trap flag и получает single-step events, пока не исчерпает trace_steps. Значение по умолчанию ограничено, максимальный рекомендованный предел - 256 samples.

Смысл trace: показать, куда начало двигаться исполнение после попадания в handler. Это полезно как evidence для ручного анализа: реверсер может увидеть, остается ли выполнение рядом с handler, уходит ли в другой участок модуля, идет ли в runtime library или в неожиданный executable region.

Ограничение: это не полноценная трассировка всего control flow и не proof graph RtlVirtualUnwind. Trace не реконструирует всю семантику обработки исключения, не восстанавливает все stack frames и не объясняет каждое изменение RSP/RIP. Это компактный runtime artifact, который помогает реверсеру начать проверку конкретного маршрута.

16. Static/dynamic correlation в текущей версии

Correlation означает связывание статических и динамических фактов в один вывод. В текущем проекте корреляция сделана консервативно и крупно, а не как детальный per-function proof graph. Это важно честно говорить: проект уже соединяет слои, но не строит полную историю каждого exception route.

Текущая модель корреляции опирается на два типа связок. Первая: есть подозрительный handler-related static context и dynamic увидел handler/VEH/VCH/UEF hit. Это повышает вес случая, потому что handler не только подозрительно выглядит в metadata, но реально получил управление.

Вторая: есть static placement anomalies и dynamic увидел runtime function-table activity. Это важно, потому что статическая странность в размещении metadata получает runtime-контекст: процесс действительно меняет unwind/function map во время выполнения.

Чего корреляция пока не делает: она не говорит "exception RIP попал в RuntimeFunction N, он указывает на UnwindInfoAddress X, handler H получил управление, после handler RIP стал Y" для каждой функции. Такой graph был бы сильным будущим улучшением, но текущая версия уже полезна как evidence collector и структурный analyzer.

17. Что dynamic может подтвердить по JIT/runtime tables

Ситуация	Что проект способен подтвердить
Target вызывает RtlAddFunctionTable	Будет событие rtl_add_function_table, счетчик rtl_add_function_table_calls увеличится, сохранятся table pointer, entry count и base address.
Target вызывает RtlInstallFunctionTableCallback	Будет событие rtl_install_function_table_callback, счетчик rtl_install_function_table_cb_calls увеличится, сохранятся table id, base, length и callback.
JIT/runtime создает код и регистрирует таблицу через эти API	Проект увидит системный след регистрации. Корректная формулировка: обнаружена runtime registration of function table, типичная для JIT/runtime-generated code.
JIT существует, но данный путь не исполнился	Проект ничего не увидит в этой dynamic-сессии. Это не false negative в строгом смысле теста, если сценарий не достиг поведения; это ограничение dynamic coverage.
Runtime использует другой API	Текущая версия может не увидеть регистрацию. Это кандидат на улучшение: добавить breakpoints на дополнительные ntdll APIs, например growable function table family.
Таблица зарегистрирована, но ее contents повреждены	Текущая dynamic-часть фиксирует факт и arguments регистрации, но не выполняет глубокий read/parse каждой runtime table entry из памяти процесса.

18. Что dynamic не делает сейчас

- Не выполняет универсальное обнаружение любого JIT. Он обнаруживает конкретные наблюдаемые API events и handler hits.
- Не читает и не валидирует каждую RUNTIME_FUNCTION entry внутри runtime table из remote process memory.
- Не дизассемблирует JIT-generated code и не сверяет его prolog с динамически зарегистрированным UNWIND_INFO.
- Не строит полный per-function route: exception RIP -> static RuntimeFunction -> UnwindInfo -> handler -> post-handler RIP для каждой функции.
- Не обходит anti-debugging. Target запущен под debugger-ом, поэтому защищенный образ может менять поведение.
- Не гарантирует покрытие неисполненных веток. Dynamic видит только то, что произошло при данном запуске, аргументах, окружении и timeout.
- Не поддерживает dynamic на macOS/Linux, потому что collector построен на Windows Debug API.

19. Почему это не делает проект бесполезным

Эти ограничения не обнуляют ценность проекта. Они определяют его класс: статический структурный analyzer с dynamic evidence collector. Для реверсера это полезно, потому что самый трудоемкий ручной слой - безопасное чтение .pdata/.xdata, проверка адресации, CountOfCodes, tail, CHAININFO, handler placement, reuse/dominance и фиксация runtime events - выполняется автоматически и воспроизводимо.

Полная семантическая реконструкция exception-based control flow потребовала бы дизассемблера, state tracking, stack modeling, RtlVirtualUnwind-like simulation, remote memory parser для runtime tables и набора targeted inputs. Это уже следующий уровень продукта. Текущий проект не делает вид, что достиг этого уровня, но закладывает корректную карту и evidence format, на которые этот уровень можно поставить.

Для защиты правильная позиция такая: проект закрывает фундаментальную структурную часть и добавляет динамическую проверку ключевых runtime-сигналов. Он не является полноценным malware sandbox, decompiler или universal JIT analyzer. Его сила - узкая, но глубокая зона Windows x64 EH/Unwind

metadata.

20. Report: какие данные дает static

Блок отчета	Что можно получить
PE/sections	Базовые сведения об image, секциях, permissions, mapping. Это нужно, чтобы понять, где лежат .pdata/.xdata/handler и насколько placement нормален.
RUNTIME_FUNCTION table	Для каждой записи: index, begin_rva, end_rva, size, begin section, unwind_rva, unwind section, parse status, error, summary.
UNWIND_INFO summary	Version, flags, prolog size, CountOfCodes, fixed stack bytes, frame register/offset, saved registers, handler RVA, handler section, chain status, anomalies.
Static heuristic matrix	Все признаки, их counters, explanation, whether triggered. Это важно для комиссии: проект не скрывает признаки, которые не сработали.
Findings	Структурированные срабатывания: code, family, severity, affected count, representative RVA, message, evidence block.
Assessment	Score, confidence, static evidence level, runtime evidence level, final case strength, manual review flag.
Manual validation steps	Что реверсер должен проверить дальше в Ghidra/PE viewer/debugger. Это не бытовые инструкции, а научно оформленная процедура верификации.

21. Report: какие данные дает dynamic

Поле/счетчик	Смысл
enabled	Был ли запрошен dynamic mode.
ran	Стартовала ли dynamic-сессия. На macOS/Linux при Windows-only collector ran будет false.
ok	Завершилась ли сессия без критической ошибки instrumentation/debug loop.
timed_out	Был ли достигнут timeout. Если да, отсутствие событий нельзя трактовать слишком сильно.
target_exited / target_exit_code	Завершился ли target естественно и с каким кодом.
instrumentation_complete	Смог ли инструмент полно поставить и сопровождать breakpoints.
exceptions_seen	Сколько обычных exception events увидел debugger.
first_chance_seen / second_chance_seen	Стадии exception flow. Нужны для различения controlled exception handling и аварийного поведения.
unwind_handlers_discovered	Сколько уникальных handler RVA найдено static-слоем в UNWIND_INFO.
unwind_handler_bp_set	Сколько breakpoints на эти handler-ы удалось поставить.
unwind_handler_hits	Сколько раз исполнение реально дошло до handler-ов из UNWIND_INFO.
veh/vch/uef counters	Регистрации и попадания в vectored/unhandled handlers.
rtl_add_function_table_calls	Сколько раз наблюдался RtlAddFunctionTable.
rtl_install_function_table_cb_calls	Сколько раз наблюдался RtlInstallFunctionTableCallback.
events	Список конкретных DynamicEvent с thread id, address, type, arguments.
trace	Короткий RIP trace после первого handler hit.

22. Как объяснить комиссии: static

Если спрашивают "что делает static", ответ должен быть не "парсит PE". Парсинг - это только техническая операция. Static строит проверяемую карту Windows x64 EH/Unwind metadata: из PE headers и section mapping он переходит к Exception Directory, затем к RUNTIME_FUNCTION entries, затем к UNWIND_INFO, затем к handler/chain/placement/reuse/dominance и формирует findings.

На защите можно сказать: static анализирует договор, который файл предъявляет Windows unwinder-у. Если этот договор поврежден, неоднозначен или подозрительно организован, проект фиксирует конкретное место и объясняет, почему это важно для stack unwinding и exception dispatch.

Важно подчеркнуть, что static является детерминированным для данного файла. При одинаковом input он выдаст одинаковый результат. Это отличает его от dynamic, где результат зависит от execution path. Поэтому static - основной слой покрытия, а dynamic - слой runtime validation.

23. Как объяснить комиссии: dynamic

Если спрашивают "что делает dynamic", ответ: dynamic запускает target на Windows x64 под управлением Windows Debug API и собирает runtime evidence по exception-related behavior. Он не просто запускает программу. Он ставит breakpoints на системные API регистрации exception handlers и dynamic function tables, ставит breakpoints на handlers из статически найденных UNWIND_INFO, считает исключения, различает first-chance и second-chance, фиксирует handler hits и короткий RIP trace.

Если спрашивают про JIT: dynamic может обнаружить появление runtime unwind/function map, если JIT или runtime регистрирует ее через RtlAddFunctionTable или RtlInstallFunctionTableCallback. В отчете будут аргументы вызова: pointer/id, entry count, base address, length, callback. Это показывает, что .pdata из файла не является полной картой для данного процесса.

Если спрашивают, почему dynamic не основной слой: потому что dynamic не может гарантировать выполнение всех веток target-a. Если поведение зависит от input, времени, окружения, anti-debugging или скрытого trigger-a, один запуск может его не показать. Поэтому научно корректная архитектура - static для полного чтения файла и dynamic для подтверждения исполнившихся runtime facts.

24. Готовый ответ про runtime tables и JIT

Формулировка для защиты:

```
■ x64 Windows ■■■■■■■■■■■■ .pdata ■■ ■■■■■■■■■■■■ ■■■ unwind-■■■■■■■■■■■ ■■■■■■■■■. ■■■■■
 ■■■■■■■■■ ■■■■■■■ ■■■■■■■■■■■■■ Exception Directory. ■■ ■■■■■■ ■■■■■■■■■■■ ■■■■■■■■ ■■■■■■
 ■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ function table ■■■■ ■■■■■, ■■■■■■■■■■■■ ■■■■ ■■■■■■■■■■■
■■■■■■ runtime-■■, ■■■■■■■■■ ■■■■ JIT ■■■■ generated code. ■■■■■■■■■■■■■■ ■■■■■■ ■■■■■■■■ ■■
■■■■■■■■ ■■■■■■■ ■■■■■ ■■■■■■ ■■■■■■■■■■■: ■■■■ ■■■■■■■ breakpoints ■■ RtlAddFunctionTable ■ R
tlInstallFunctionTableCallback, ■■■■■■■■■■■ target ■■■■ Windows Debug API ■ ■■■■■■■■■■■ ■■■■■ ■■
■■■■■ ■■■■■■■ ■ ■■■■■■■■■■■■■■. ■■■■■■■■ ■■■■■■■ ■■ ■■■■■■■■■ "■■■■■■■■■■■■■■ ■■■■■■■■■■■ ■
■■■■■ JIT", ■ ■■■■■■■■■ ■■■■■■ ■■■■■■■ ■■■■■■■■■■■■■: ■■■■■■■■■■■■■ runtime-■■■■■■■■■■■■■ ■■
■■■■■■■■■■■■■■ unwind/function tables ■■■■■■ ■■■■■■■■■■■■■■■■■ Windows API ■ ■■■■■■■■■■■, ■■■■
■■■■■■■■■■■■■ .pdata ■ ■■■■■ ■■■■■■■ ■■ ■■■■■■■■■ ■■■■■■■ ■■■■■■■.
```

25. Типовые сценарии и как проект их видит

Сценарий	Static	Dynamic
Нормальная PE x64 .pdata/.xdata	Читает entries, UNWIND_INFO, показывает отсутствие findings или weak/no case.	Не обязан находить события, если исключения и runtime registrations не происходят.
Поврежденный CountOfCodes	Останавливает глубокий разбор конкретной UNWIND_INFO, фиксирует parse failure/diagnostic.	Может не проявиться, если функция не исполняется и exception route не достигается.
Handler RVA в writable section	Дает placement finding и manual review.	Может подтвердить handler hit, если исполнение реально дошло до handler-а.
Массовый reuse одного UnwindInfoRVA	Дает reuse/dominance context или finding по порогам.	Сам reuse не обязан проявиться, но handler hits/runtime events усиливают case.
CHAININFO loop	Static обнаруживает loop/depth anomaly и показывает chain path.	Dynamic увидит runtime последствия только при достижении соответствующего пути.
JIT/runtime table	Static видит только PE file map и может не увидеть generated code.	Видит RtlAddFunctionTable/RtlInstallFunctionTableCallback, если вызов произошел в сессии.

26. Почему dynamic не может быть "всевидящим"

Dynamic-анализ всегда зависит от достижимости поведения. Если код регистрации runtime table находится после условия, сетевого ответа, лицензии, таймера, проверки debugger-а или специального input, запуск без этого условия не покажет событие. Поэтому отсутствие event в одном dynamic report нельзя автоматически превращать в утверждение "в программе нет runtime tables".

Кроме достижимости есть вопрос механизма. Текущий проект ставит breakpoints на конкретный набор API. Если target использует неподдержанный API, обходит импорт, делает прямой переход в ntdll stub необычным способом или применяет anti-debugging, событие может быть пропущено. Это нормальная граница для lightweight debugger evidence collector.

В научной формулировке dynamic дает наблюдаемые факты, а не абсолютную полноту. Поэтому в отчете важны counters качества сессии: timed_out, instrumentation_complete, dropped_events, breakpoint failures. Они не являются деталями ради деталей; они говорят, насколько сильным можно считать отсутствие или наличие dynamic evidence.

27. Связь с ручной работой реверсера

Реверсер вручную сначала вынужден установить адресную модель: где ImageBase, где RVA, где file offset, какие секции покрывают нужные диапазоны. Проект автоматизирует этот слой и снижает риск перепутать VA/RVA/file offset. Это особенно важно в EH/Unwind metadata, потому что одна ошибка адресации приводит к чтению другой структуры и ложным выводам.

Дальше реверсер вручную читает RUNTIME_FUNCTION entries и проверяет, отсортированы ли они, не пересекаются ли функции, есть ли gaps, куда указывает UnwindInfoAddress. Проект делает это массово по всему образу, а не по одному адресу. Для защиты это можно формулировать так: проект масштабирует ручной ритуал проверки одного адреса на весь PE-файл.

Затем реверсер вручную разбирает UNWIND_INFO: header, slots, дополнительные operands, padding, tail, handler/chain. Проект автоматизирует именно те места, где человек чаще всего ошибается: CountOfCodes, multi-slot UWOP, padding, flags/tail selection, chain depth/loop.

Наконец, реверсер вручную запускает target, ставит breakpoints на handlers и runtime APIs, смотрит exception events и handler hits. Dynamic автоматизирует эту часть частично: он ставит заранее известные breakpoints, ведет counters, сохраняет structured events и trace. Полноценная интерпретация маршрута остается за специалистом, но сбор фактов становится воспроизводимым.

28. Как правильно называть архитектуру

Архитектурно проект является монолитным CLI-инструментом с разделенными внутренними слоями: PE parser, unwind parser, heuristics engine, dynamic collector, report generator. Это не микросервисная система. Для ядра анализа выбран монолитный формат, потому что обработка одного binary требует тесного обмена структурами между parser, heuristics и reports, а запуск в CLI удобен для reverse engineering workflow.

Разделение static и dynamic в схеме не означает два отдельных сервиса. Это два логических слоя внутри одного инструмента. Static слой можно выполнить на любой ОС для PE x64 файла. Dynamic слой требует Windows, потому что использует Windows Debug API и запускает target process. Для будущего front/back/database можно оставить ядро как CLI/библиотечный компонент и оборачивать его внешним backend-слоем, не ломая analyzer.

29. Реалистичные улучшения dynamic

Улучшение	Что даст
Добавить RtlAddGrowableFunctionTable/RtlGrowFunctionTable/RtlDeleteGrowableFunctionTable	Расширит покрытие runtime function table mechanisms, которые текущий список API может не видеть.
ReadProcessMemory для runtime table entries	Позволит не только фиксировать факт RtlAddFunctionTable, но и читать entries, проверять Begin/End/UnwindInfoAddress в памяти процесса.
Связка exception RIP с static RuntimeFunction	Позволит в dynamic report писать: exception address попал в конкретный static function range и связан с конкретным finding.
Перехват RtlLookupFunctionEntry/RtlVirtualUnwind	Позволит наблюдать, как Windows lookup/unwind реально использует таблицы. Это сильнее, но сложнее и чувствительнее к версии системы.
Disassembly prolog verification	Добавит семантический слой: сравнение UWOP с байтами/инструкциями пролога. Это устранит часть false positives и даст более объяснимые findings.
Targeted scenarios/input profiles	Увеличит покрытие dynamic: разные аргументы, environment, controlled exception triggers, несколько запусков.
Per-function proof graph	Соберет идеальный отчет: PE -> RuntimeFunction -> UNWIND_INFO -> exception event -> handler hit -> post-handler RIP trace.

30. Итоговая позиция

Static в проекте - это не поверхностная печать таблицы. Это безопасный структурный анализ PE x64 EH/Unwind metadata с проверкой адресации, формата, sections, RUNTIME_FUNCTION ranges, UNWIND_INFO, slots, padding, tail, CHAININFO, handler placement, reuse/dominance и evidence model.

Dynamic в проекте - это не полный runtime decompiler и не универсальный JIT detector. Это Windows-only dynamic evidence collector: он запускает target под Debug API, наблюдает exception events, handler registrations/hits и runtime function table API. Его особенно важная возможность для твоего вопроса: он может зафиксировать появление runtime function tables через RtlAddFunctionTable и RtlInstallFunctionTableCallback, то есть показать, что .pdata из файла не вся unwind-карта процесса.

Идеальная формулировка для защиты: проект автоматизирует статическую реконструкцию EH/Unwind-карты PE x64 и дополняет ее динамической валидацией ключевых runtime-сигналов. Он дает реверсеру структурированные данные и evidence, снижает риск ручных ошибок и показывает, какие выводы основаны на файле, а какие подтверждены выполнением.

31. Приложение: dynamic-механика без темных пятен

Эта часть добавлена специально для защиты, потому что вопросы по dynamic обычно самые опасные. Static можно показать таблицами: PE, sections, RUNTIME_FUNCTION, UNWIND_INFO. Dynamic сложнее, потому что там участвуют живой процесс, debugger events, breakpoints, регистры потока и runtime API. Поэтому ниже каждый механизм раскрыт не как список терминов, а как причинно-следственная цепочка.

Главная формула dynamic-слоя: analyzer не заглядывает мистически внутрь процесса. Он создает наблюдаемые точки контроля. Для API он ставит breakpoint на начало функции. Для handler-ов из UNWIND_INFO он ставит breakpoint на вычисленный runtime address. Для исключений он слушает Windows Debug API event stream. В результате отчет содержит не вывод в стиле "кажется, был JIT", а конкретные события: какой API был достигнут, в каком потоке, на каком адресе, с какими аргументами.

32. Breakpoint: что фактически происходит

Software breakpoint обычно реализуется через инструкцию INT3. Analyzer временно заменяет первый byte целевого адреса на INT3. Когда поток target-а доходит до этого адреса, процесс генерирует breakpoint exception. Windows передает это событие debugger-у как EXCEPTION_DEBUG_EVENT.

После попадания в breakpoint нельзя просто продолжить выполнение. RIP после INT3 указывает на следующий byte, поэтому analyzer обязан вернуть RIP на адрес breakpoint. Затем нужно восстановить оригинальный byte, выполнить оригинальную инструкцию один раз и снова поставить breakpoint. Для этого используется single-step: analyzer включает trap flag, получает EXCEPTION_SINGLE_STEP после одной инструкции и re-arm-ит breakpoint.

Эта схема объясняет, почему в отчете есть поля про breakpoint_set_attempts, breakpoint_set_failures, pending_rearm_failures и breakpoint_rearm_failures. Они нужны не для украшения, а для оценки качества dynamic-инструментации. Если breakpoint не поставился или не был восстановлен, отсутствие события по этому адресу нельзя считать сильным отрицательным доказательством.

33. Почему API-аргументы берутся из RCX/RDX/R8/R9

Windows x64 calling convention передает первые четыре integer/pointer аргумента через RCX, RDX, R8 и R9. Analyzer ставит breakpoint на entry point API. Это означает, что поток остановлен в момент входа в функцию, до того как функция исполнила собственный prolog. На этом шаге аргументы еще находятся в регистрах согласно соглашению вызова.

Для RtlAddFunctionTable интерпретация такая: RCX - pointer на массив FunctionTable; RDX - EntryCount; R8 - BaseAddress. Поэтому событие rtl_add_function_table в отчете несет конкретный смысл: процесс передал Windows адрес runtime table, число записей и базовый адрес, относительно которого записи будут интерпретироваться.

Для RtlInstallFunctionTableCallback интерпретация такая: RCX - TableIdentifier; RDX - BaseAddress; R8 - Length; R9 - Callback. Это значит, что процесс не просто добавляет готовый массив, а регистрирует callback-модель для диапазона адресов. Для JIT/runtime-generated code это особенно важно: unwind-информация может предоставляться лениво через callback.

API/event	RCX / arg0	RDX / arg1	R8 / arg2	R9 / arg3
RtlAddFunctionTable	FunctionTable pointer	EntryCount	BaseAddress	не используется проектом
RtlInstallFunctionTableCallback	TableIdentifier	BaseAddress	Length	Callback
AddVectoredExceptionHandler	First/order	Handler VA	не используется	не используется
AddVectoredContinueHandler	First/order	Handler VA	не используется	не используется

API/event	RCX / arg0	RDX / arg1	R8 / arg2	R9 / arg3
SetUnhandledExceptionFilter	Handler VA	не используется	не используется	не используется

34. Почему обычные exceptions продолжаются как not handled

Когда target генерирует обычное исключение, analyzer получает EXCEPTION_DEBUG_EVENT. Если это не наш INT3 breakpoint и не single-step, событие отражает реальную работу target-a. В этом случае analyzer записывает exception_code, address и first_chance/second_chance, но обычно возвращает DBG_EXCEPTION_NOT_HANDLED.

Смысл DBG_EXCEPTION_NOT_HANDLED: debugger говорит Windows, что он не обработал исключение за target. Тогда система продолжает штатный exception dispatch: VEH, frame-based handlers, language-specific handlers, unhandled filter. Если бы analyzer отвечал DBG_CONTINUE на каждое исключение, он мог бы искусственно подавить исключение и разрушить реальный маршрут обработки.

Поэтому dynamic-слой в проекте старается быть наблюдателем, а не участником логики target-a. Он активно вмешивается только в свои instrumentation breakpoints, потому что их нужно технически обслуживать. Для обычных exception events он сохраняет evidence и позволяет Windows продолжить нормальную обработку.

35. First-chance и second-chance как evidence

First-chance exception означает первую возможность debugger-a увидеть исключение. Это не обязательно ошибка. Многие программы, рантаймы и защитные механизмы намеренно используют исключения как часть управления или диагностики. Поэтому first-chance сам по себе чаще является context signal, а не сильным finding.

Second-chance exception появляется, когда обычная обработка не справилась и исключение снова дошло до debugger-a. Это сильнее: процесс может быть близок к аварийному завершению. В отчете separate counters first_chance_seen и second_chance_seen позволяют не смешивать controlled exception flow и crash-like behavior.

Для защиты можно объяснять так: dynamic не только считает exceptions, но и различает стадию обработки. Это важно, потому что один и тот же exception code может быть штатным сигналом в first-chance сценарии и аварийным симптомом во second-chance сценарии.

36. RtlAddFunctionTable: как объяснить одно событие

Если в dynamic report есть событие rtl_add_function_table, правильная интерпретация должна идти по шагам. Первый шаг: процесс действительно дошел до entry point RtlAddFunctionTable, потому что сработал breakpoint. Второй шаг: analyzer прочитал регистры потока и сохранил pointer на table, count и base. Третий шаг: из этого следует, что target попросил Windows добавить runtime function table.

```
type = rtl_add_function_table
arg0 = FunctionTable pointer
arg1 = EntryCount
arg2 = BaseAddress
meaning = runtime unwind/function map registration observed
```

На защите нельзя автоматически говорить "обнаружен вредоносный JIT". Надо говорить: "обнаружена регистрация runtime function table; это механизм, используемый JIT, runtime-generated code, языковыми runtime-ами и некоторыми защитными системами. Для нашего проекта важно, что static .pdata больше не является полной картой unwind-информации процесса".

37. RtlInstallFunctionTableCallback: почему это еще интереснее

RtlInstallFunctionTableCallback указывает на более динамическую модель. Вместо того чтобы передать готовый массив RUNTIME_FUNCTION entries, процесс регистрирует callback для диапазона адресов. Когда Windows понадобится function entry для адреса в этом диапазоне, callback может предоставить ее.

Для реверсера callback-модель важна потому, что данные могут быть вычисляемыми, ленивыми или зависящими от состояния runtime-а. Static по файлу такую карту не увидит. Dynamic хотя бы фиксирует факт регистрации callback-а, base, length и callback address. Это уже доказательство, что runtime unwind-инфраструктура в процессе активна.

Граница текущего проекта: он фиксирует регистрацию и аргументы, но не вызывает callback и не реконструирует все entries. Это честная граница. Следующий уровень улучшения - безопасно читать remote memory или наблюдать RtlLookupFunctionEntry/RtlVirtualUnwind, чтобы понять, какие function entries реально возвращаются для конкретных addresses.

38. Связь dynamic с JIT: что можно доказать строго

Строго доказать "это JIT" только по RtlAddFunctionTable нельзя. JIT - это причина или архитектурная модель генерации кода. RtlAddFunctionTable - это системный след регистрации unwind map. Один и тот же след может появиться у JIT, у runtime-generated stubs, у интерпретатора, у защитного packer-a или у легального runtime-a.

Что проект доказывает строго: в данной dynamic-сессии target зарегистрировал runtime function table или callback table через поддерживаемый API. Что проект предполагает с осторожностью: такая регистрация совместима с JIT/runtime-generated code и означает, что static PE map неполна. Что должен делать реверсер дальше: смотреть memory region, permissions, call stack, module ownership, surrounding code и связь с exceptions/handler hits.

Это и есть профессиональная позиция. Analyzer не подменяет реверсера, а дает ему доказательства с правильной степенью силы. Событие runtime table registration - сильный факт. Причина события - интерпретация, которая требует контекста.

39. Почему dynamic не видит все возможные runtime tables

Есть несколько причин. Первая - execution coverage. Если код регистрации находится в ветке, которая не была выполнена, API не будет вызван в наблюдаемой сессии. Вторая - API coverage. Текущая версия ставит breakpoints на RtlAddFunctionTable и RtlInstallFunctionTableCallback, но не на все возможные родственные механизмы вроде growable function table family. Третья - anti-debugging и environment sensitivity: target может менять поведение под debugger-ом.

Четвертая причина - process boundary. Флаг DEBUG_ONLY_THIS_PROCESS означает, что analyzer отлаживает этот процесс. Если поведение вынесено в дочерний процесс, отдельный service или injected remote process, текущая dynamic-сессия может его не покрыть. Пятая причина - timeout: событие могло произойти позже настроенного лимита.

Поэтому отсутствие dynamic-события нужно формулировать строго: "в данной сессии событие не наблюдалось". Это не равняется глобальному доказательству отсутствия JIT или runtime function tables. Такой язык защищает проект от неправильных научных претензий.

40. Как связать static finding и dynamic evidence в рассказе

Хороший устный разбор строится как цепочка: static нашел подозрительный участок metadata; dynamic проверил, проявилось ли runtime-поведение, связанное с этим участком. Например, static показывает handler_dominance и handler placement. Dynamic показывает unwind_handler_hit или VEH/UEF hit. Тогда вывод сильнее: handler не только подозрительно описан в файле, но реально получил управление.

Другой пример: static показывает необычное placement metadata или подозрения вокруг executable/writable sections. Dynamic показывает RtlAddFunctionTable. Тогда вывод: в файле есть признаки нестандартной организации metadata, а при выполнении process дополнительно регистрирует runtime unwind map. Это не доказывает вредоносность, но дает реверсеру сильный повод продолжать ручную проверку.

Важно: текущая корреляция coarse-grained. Она не строит полный proof graph для каждой функции. Это можно объяснять как осознанный этап проекта: сначала надежный structural analyzer и runtime evidence collector, затем в будущем per-function graph и semantic prolog verification.

41. Как testing должен доказывать dynamic-возможности

Для dynamic runtime table проверки тестовый бинарник должен гарантированно вызвать нужный API. Нельзя проверять эту возможность на случайном PE и удивляться, что события нет. Нужен controlled target: один сценарий вызывает RtlAddFunctionTable, другой вызывает RtlInstallFunctionTableCallback, третий регистрирует VEH/VCH/UEF, четвертый провоцирует handler hit или controlled exception.

Минимальные критерии успешного dynamic-теста: dynamic enabled = true; ran = true; ok = true; timed_out = false; target_exited = true; instrumentation_complete не содержит критического провала; event list содержит ожидаемый event type; соответствующий counter больше нуля; arguments не пустые и соответствуют сценарию.

Такой подход к тестированию важен для комиссии. Он показывает, что проверяется не "запускается ли программа", а конкретная функция проекта: способность наблюдать runtime evidence, которое static по файлу увидеть не может.

42. Готовые формулировки ответов комиссии

Вопрос	Ответ
Может ли проект проверить JIT?	Он проверяет не абстрактный JIT, а системный след JIT/runtime-generated code: регистрацию runtime function tables через RtlAddFunctionTable или RtlInstallFunctionTableCallback. Если такой путь выполнен, событие будет в dynamic report.
Почему не сказать, что он видит любой JIT?	Потому что dynamic зависит от execution path и списка поддерживаемых API. Научно корректно говорить: обнаруживаются поддерживаемые runtime function table registrations в наблюдаемой сессии.
Зачем dynamic, если есть static?	Static видит файл и статическую .pdata. Dynamic видит процесс, handler hits, exception events и runtime registrations, которых может не быть в файле.
Почему отсутствие события не доказывает отсутствие JIT?	Потому что ветка могла не выполниться, target мог использовать другой API, мог сработать anti-debugging или timeout. Отсутствие dynamic evidence - это ограниченный отрицательный результат конкретной сессии.
Проект парсер или analyzer?	Parser - нижний слой. Проект также имеет heuristics, scoring/confidence, reports и dynamic collector, поэтому корректнее называть его системой структурного анализа и динамической валидации EH/Unwind metadata.

43. Короткая финальная формула

```
Static answers: what unwind/exception map the PE file declares.
Dynamic answers: what exception/unwind-related runtime events actually occurred.
Runtime table events answer: whether the process extended the unwind map after load.
```

По-русски: static строит карту обещаний файла, dynamic собирает факты живого исполнения. Если dynamic видит RtlAddFunctionTable или RtlInstallFunctionTableCallback, значит в runtime появилась дополнительная function table, и .pdata из PE-файла не является всей картой unwind-информации процесса. Именно это и нужно уверенно говорить на защите.